

Prompt Documentation — sample input, sample output & step-by-step usage guide

So any FA member can replicate the full workflow independently, using only the files in this package. No installation is needed for the dashboard (a browser is enough); Python is only needed if you also want to run the parsers.

A. Sample input (included in this package)

File	Location	What it is
AP_PAYMENT_SAMPLE_SANITIZED.xlsx	07_Demo_Data/	A parsed AP batch: 18 payment groups · 6 entities · 5 currencies, in the parser's output format (PAYMENT_LIST + PARSE_CONTROL sheets). Two rows are deliberately incomplete (one missing value date, one missing account) so the validation workflow has something to catch. 100% synthetic — no real vendor, account or amount.
BANK_RESULT_SAMPLE.xlsx	07_Demo_Data/	The matching bank creation result: 13 exact matches, 1 amount difference (+100), 1 wrong account, 1 SWIFT branch-code case.

For the parser step, any real AP payment-list Excel works (place files in an `input_ap/` folder); the sample above already IS parser output, so you can skip straight to Step 2.

B. Step-by-step usage guide

Step 0 — Open the tool. Double-click `03_Source_Code/dashboard/index.html` (or open <https://tultc4.github.io/treasury-payment-dashboard/>). You should see the **Control Tower** home, empty.

Step 1 — (Optional) Parse raw AP files.

```
cd 03_Source_Code/ap_parser
pip install -r requirements.txt
python ap_payment_template_parser_fixed_v4.py --input <folder of AP files> --output AP_PAYMENT_TEMPLATE_OUTPUT.xlsx --date 2026-08-19
```

✓ *Expected:* console prints one reconciliation line per file and ends with `PARSE CONTROL: n/n file(s) reconciled`. The output workbook has `PAYMENT_LIST + PARSE_CONTROL` sheets.

Step 2 — Import the batch. Go to **Payment List** → click the  upload icon → choose `AP_PAYMENT_SAMPLE_SANITIZED.xlsx`.

✓ *Expected:* toast `Imported AP summary ... (18 total payment lines) · parse control 1/1 file(s) reconciled`; the table shows 18 rows with Payment Bank / Method pre-filled.

Step 3 — Fix what validation found. A red **"Needs manual fix · 1"** box appears → click **Go to next fix** → the row missing its value date flashes.

✓ *Fix it:* in the same card, set **Set Value Date** = `2026-08-19` → **Apply Filtered** → read the confirmation dialog (scope + consequences) → OK. An  **Undo last bulk** button appears.

Step 4 — Export ERP files. Go to **ERP Export** → every entity card shows READY → click **Download CSV** on any entity.

✓ *Expected:* a CSV named `ERP_UPLOAD_<ENTITY>_....csv` with the header:

```
ORG_ID,SUPPLIER_NUM,AP_INVOICE_NUMBER,EFORM_NUMBER,PAYMENT_AMOUNT,DESCRIPTION,PAYMENT_METHOD,CHARGES_INDICATOR,ORDERING_PARTY,PURPOSE_CODE,,Value
date,CREATED_BY
```

and one row per payment (e.g. `...,DEMOINV26001,FA-PM260801,57728.70,...`). A payment with a blocked reference, missing field or zero amount can never appear in this file.

 **Duplicate-payment control:** the references you just exported are now in the permanent

reference history register (Admin → Payment reference history). If you re-import the same

AP file, those payments are flagged `DUPLICATE_IN_BANK` and blocked from export — that is the

control working. To repeat this demo from scratch, click **Clear register...** in Admin first.

Step 5 — Reconcile the bank result. Go to **Bank Creation** → upload icon → `BANK_RESULT_SAMPLE.xlsx`.

✓ *Expected:* summary chips read **Payments in result 15 · Matched (CREATED) 13 · Diff to review 2**. Click the red **Diff to review** chip → it jumps to the first difference and the Difference column names the exact field (`Amount: bank ... vs ours ...`). Click again → second difference (wrong account). Type the bank's account into that row's Vendor Bank Account cell → the Match badge flips to **TRUE** instantly. A  appears on one SWIFT ending `B01` (branch-code review case).

Step 6 — Approval email. Tick **Select all matched** → click the  icon.

✓ *Expected:* a drafted email — subject `[Oversea] - Approval Payment - <date> - Citi bank`, body with per-entity counts, and a **Download attachment (.xlsx)** holding a PAYMENT LIST sheet and a per-currency SUMMARY sheet.

Step 7 — Debit confirmation & cash sufficiency. Go to **Bank Debit / Cash Position** (with your own T-1 statement file parsed by `03_Source_Code/bank_statement_parser/`).

✓ *Rule to verify:* a payment is marked DEBIT CONFIRMED only when its transaction appears in the statement — a balance movement alone never confirms anything.

Step 8 — Exceptions & audit. Go to **Exceptions** → give one item an Owner and set Status = Resolved (the bell count drops). Go to **Admin** → **Audit log**.

✓ *Expected:* every action you just performed is listed — import, parse control result, bulk update, ERP export, inline edit, exception status — each with who + when.

C. Sample output summary (what a correct replication looks like)

Checkpoint	Expected value
Import toast	18 payment lines · parse control 1/1 reconciled
Needs manual fix	1 (missing value date)
ERP validation	0 blocking issues after Step 3
Bank reconciliation	13 CREATED · 2 DIFF · 1 SWIFT ⚠
After inline fix	DIFF count drops to 1; edited row = TRUE
Audit log	≥ 6 events, newest first, all by <code><operator name></code>

D. Using it with your own data

- Put your entities' AP Excel files in one folder and run Step 1 with your run date — the parser detects entity from the filename and reconciles every row in `PARSE_CONTROL`.
- Entity/bank/method rules and account masters are data, not code — see `04_Architecture/ARCHITECTURE.md` and the Admin screen. Adding an entity or bank is configuration.
- Everything runs locally in the browser; no payment data ever leaves the machine.